

Piece Generation

We developed an algorithm to break the octagon into individual puzzle pieces. We rasterized the octagon into a dense grid of small triangles and kept only those whose centers were inside of the octagon's boundary. We then scattered a configurable number of seed points throughout the octagon, with the number of seeds corresponding to the desired number of puzzle pieces. Each triangle was assigned to the closest seed based on its centroid, and all triangles belonging to the same seed point were merged into a single mesh. This produces random, organic shaped pieces with no gaps each time the program runs.

Random Piece Placement

We implemented a brute force algorithm to randomly position each puzzle piece around the playing area. Each piece has a bounding box, and random positions are generated until 3 conditions have been met: the piece is inside the play area, doesn't overlap the octagon, and doesn't overlap any previously placed pieces. Once a valid position is found, we save its bounding box for future checks.

Mouse Controls

We implemented mouse controls using a raycaster, mouse vector, and hold, move, and release events. A ray is sent from the camera to determine which piece was clicked, while another ray determines where the mouse intersects the puzzle plane, allowing us to drag the selected piece. When a piece is within 0.15 units of its correct position, it automatically snaps into place and becomes locked.

Above and Beyond - Sound Effects

We added sound effects to provide feedback throughout the game. We included sounds for starting the game, clicking pieces, incorrect placements, correct placements, and completing the puzzle. Each sound is triggered when its corresponding event occurs.

Above and Beyond - Victory Animation

We created a victory animation that activates once every puzzle piece has been correctly placed. The animation displays a “You Win!” message and sparkles while the octagon spins. This provides visual feedback that the puzzle has been successfully completed.

Challenges

The biggest challenge we faced was developing the puzzle piece generation algorithm, which required a significant amount of research to find an easy and efficient way to generate pieces in a way that satisfied the rubric. Another challenge we faced was preventing randomly placed pieces from overlapping, as well as ensuring that pieces did not spawn entirely or partially outside the play area. We solved that by adjusting the play area limits. Another challenge was the difference between the mouse and Three.js coordinate systems, which initially caused pieces to be slightly offset from the cursor. We resolved this by applying an offset. Finally, the victory animation was a bit challenging to implement compared to the sound because it required more logic and visual design.